

دانشگاه صنعتی امیرکبیر

دانشکده مهندسی کامپیوتر

مبانی رایانش ابری

نیمسال دوم ۱۴۰۴-۱۴۰۵

فاز دوم پروژه پایانی

استقرار روی Kubernetes، خودکارسازی با GitLab CI/CD و انتشار خروجی

استاد درس	دکتر جوادی
تدریس‌یاران	کیان پورآذر، امیرحسین متقیان شناسه (ID) تلگرام: @KiyanPourazar - @AmirH_Motaghian
تعداد نفرات گروه	گروه‌های دو نفره
پیش‌نیاز	تکمیل فاز اول (تولید لاگ و pipeline پنج مرحله‌ای MapReduce)
هدف اصلی	خارج کردن پروژه از اجرای دستی و انتقال آن به Kubernetes، راه‌اندازی GitLab و GitLab Runner درون کلاستر، و خودکارسازی کامل مسیر build، deploy، تولید لاگ، تحلیل و انتشار با CI/CD
مهلت تحویل	۲ مرداد ۱۴۰۵، ساعت ۲۳:۵۹

مورد تحویلی

نام فایل زیپ نهایی فاز دوم باید مطابق قالب زیر باشد و در آن StudentID1 و StudentID2 با شماره‌ی دانشجویی دو عضو گروه جایگزین شوند:

CC_Project_Phase2_StudentID1_StudentID2.zip

۱. هدف فاز دوم

در فاز اول، یک سامانه‌ی چندسرویس‌ی تولید لاگ ساختید، با Docker Compose آن را بالا آوردید، ترافیک تولید کردید و لاگ‌ها را با pipeline پنج‌مرحله‌ای Hadoop Streaming تحلیل کردید. خروجی نهایی آن فاز، فایل outputs/final/summary.json به همراه خروجی‌های میانی هر job بود.

هدف فاز دوم این است که همان سامانه از حالت «اجرای دستی روی یک ماشین» خارج شود و به یک محیط شبیه ابر واقعی منتقل شود. در این فاز سه تغییر اصلی رخ می‌دهد:

اجزای پروژه (سرویس‌های Nginx، backend، تولید ترافیک و pipeline MapReduce) دیگر با Docker Compose مدیریت نمی‌شوند، بلکه به شیء‌های Kubernetes مثل Job، Service، Deployment و PersistentVolumeClaim تبدیل می‌شوند؛

یک نمونه GitLab و یک GitLab Runner درون همین کلاستر Kubernetes راه‌اندازی می‌شود و مخزن پروژه داخل آن قرار می‌گیرد؛

کل مسیر ساخت، تست، استقرار، تولید لاگ، اجرای پنج job اجباری MapReduce، اعتبارسنجی و انتشار گزارش، با یک pipeline در GitLab CI/CD خودکار می‌شود و خروجی نهایی به صورت قابل مشاهده deploy می‌شود.

توجه

استفاده از GitLab.com به تنهایی جایگزین راه‌اندازی GitLab و Runner در Kubernetes نیست. هدف این فاز این است که گروه با اجرای واقعی GitLab، Runner و سرویس‌های پروژه درون Kubernetes درگیر شود. اجرای پروژه فقط با Docker Compose نیز برای این فاز کافی نیست.

۲. معماری مورد انتظار فاز دوم

در فاز دوم، همه‌ی اجزای پروژه درون یک کلاستر Kubernetes اجرا می‌شوند. مسیر منطقی درخواست‌ها همان مسیر فاز اول است، با این تفاوت که هر جزء اکنون یک Pod پشت یک Service است و GitLab Runner مسئول اجرای خودکار همه‌ی مراحل است.



اجزای مورد انتظار در این معماری عبارت‌اند از:

یک کلاستر Kubernetes (minikube، kind یا k3d) که نقش محیط ابری را بازی می‌کند؛

Deployment و Service برای سه سرویس backend فاز اول و برای Nginx به‌عنوان API Gateway؛

یک PersistentVolumeClaim مشترک برای نگهداری لاگ‌ها و خروجی‌ها بین مرحله‌ی تولید ترافیک و مرحله‌ی MapReduce؛

یک Job برای تولید ترافیک و یک Job برای اجرای pipeline MapReduce؛

یک نمونه GitLab و یک GitLab Runner که هر دو درون کلاستر اجرا می‌شوند؛

یک سرویس انتشار خروجی (report-web) که گزارش یا داشبورد نهایی را از طریق Service/NodePort/Ingress قابل مشاهده می‌کند.

۳. آماده‌سازی کلاستر Kubernetes

اولین قدم، داشتن یک کلاستر در دسترس است. استفاده از سرویس ابری پولی اجباری نیست؛ اجرای پروژه روی `minikube` یا `kind` کاملاً قابل قبول است، به شرطی که اجزای پروژه واقعاً توسط Kubernetes مدیریت شوند.

۳.۱ ساخت کلاستر

از کلاستر کوبرنتیز تمرین قبلی خود استفاده کنید یا در صورت نبود آن با استفاده از `kind` یا `minikube` یک کلاستر جدید راه اندازی کنید

۳.۳ ساخت و در دسترس قرار دادن ایمیج‌ها

سرویس‌های فاز اول و ابزار تولید ترافیک باید به ایمیج Docker تبدیل و سپس در دسترس کلاستر قرار بگیرند. می‌توانید مانند تمرین قبل آن‌ها را به رجیستری داخلی بالا آورده بفرستید یا در `kind/minikube` چون رجیستری خارجی ندارید، ایمیج‌های ساخته‌شده را مستقیماً به داخل نود کلاستر بارگذاری می‌کنید:

```
docker build -t match-service:1.0 ./services/match-service
docker build -t team-service:1.0 ./services/team-service
docker build -t stadium-service:1.0 ./services/stadium-service
docker build -t traffic-generator:1.0 ./traffic-generator

#kind:
kind load docker-image match-service:1.0 team-service:1.0\
    stadium-service:1.0 traffic-generator:1.0 --name cc-project

#minikube:
minikube image load match-service:1.0
```

نکته

چون ایمیج‌ها لوکال هستند، در `manifest`‌ها حتماً `imagePullPolicy: IfNotPresent` بگذارید؛ در غیر این صورت Kubernetes سعی می‌کند ایمیج را از رجیستری اینترنتی بکشد و با خطای `ImagePullBackOff` مواجه می‌شوید.

۴. استقرار سرویس‌های فاز اول روی Kubernetes

در این بخش، اجزایی که در فاز اول با Docker Compose اجرا می‌شدند، به Service و Deployment تبدیل می‌شوند. نکته‌ی کلیدی این است که نام Service در Kubernetes نقش همان نام سرویس در docker-compose را بازی می‌کند و از طریق DNS داخلی کلاستر قابل دسترسی است (مثلاً ۸۰۰۰:match-service).

۴.۱ Service و Deployment سرویس‌های backend

برای هر سه سرویس یک Deployment و یک Service تعریف کنید.

نکته

هر سرویس لاگ ساختاریافته‌ی اختصاصی خود را در مسیر data/service_logs/ می‌نویسد. چون این مسیر روی PVC مشترک mount شده، Job مربوط به MapReduce بعداً می‌تواند همین لاگ‌ها را بخواند. نام Service باید دقیقاً همان نامی باشد که در upstreamهای Nginx استفاده می‌کنید.

۴.۲ انتقال Nginx به ConfigMap و Deployment

فایل nginx.conf فاز اول به یک ConfigMap منتقل می‌شود. تنها تغییر محتوایی این است که آدرس upstreamها از نام کانتینرهای Compose به نام Serviceهای Kubernetes تغییر می‌کند (که در این پروژه یکسان‌اند).

```
apiVersion: v1
kind: ConfigMap
metadata: { name: nginx-gateway-conf }
data:
  nginx.conf: |
    events {}
    http {
      upstream match_up    { server match-service:8000; }
      upstream team_up     { server team-service:8000; }
      upstream stadium_up { server stadium-service:8000; }
      log_format json_logs escape=json '{"timestamp":"$time_iso8601",'
        '"request_id":"$http_x_request_id","client_ip":"$remote_addr",'
        '"client_country":"$http_x_client_country","scenario":"$http_x_scenario",'
        '"method":"$request_method","path":"$request_uri",'
```

```

    "service": "$target_service", "status_code": $status, '
    "request_time_sec": "$request_time", "user_agent": "$http_user_agent"}';
access_log /var/log/nginx/nginx_access.log json_logs;
server {
    listen 80;
    set $target_service "unknown";
    location /api/matches { set $target_service match-service; proxy_pass
http://match_up; }
    location /api/teams { set $target_service team-service; proxy_pass
http://team_up; }
    location /api/stadiums { set $target_service stadium-service; proxy_pass
http://stadium_up; }
}
}

```

سپس Deployment و Service مربوط به Nginx (با mount کردن ConfigMap روی مسیر کامل nginx.conf و mount کردن PVC برای محل لاگ):

```

apiVersion: apps/v1
kind: Deployment
metadata: { name: nginx-gateway }
spec:
  replicas: 1
  selector: { matchLabels: { app: nginx-gateway } }
  template:
    metadata: { labels: { app: nginx-gateway } }
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports: [{ containerPort: 80 }]
          volumeMounts:
            - { name: conf, mountPath: /etc/nginx/nginx.conf, subPath: nginx.conf }
            - { name: data, mountPath: /var/log/nginx }
      volumes:
        - name: conf
          configMap: { name: nginx-gateway-conf }
        - name: data
          persistentVolumeClaim: { claimName: project-data }
---

```

```
apiVersion: v1
kind: Service
metadata: { name: nginx-gateway }
spec:
  type: NodePort
  selector: { app: nginx-gateway }
  ports: [{ port: 80, targetPort: 80, nodePort: 30080 }]
```

توجه

همان نکته‌ی فاز اول این‌جا هم برقرار است: `nginx.conf` باید روی مسیر `etc/nginx/nginx.conf` نگاشت شود (با `subPath`)، نه داخل `conf.d`. اگر کل دایرکتوری را `mount` کنید فایل‌های پیش‌فرض `Nginx` پاک می‌شوند و سرویس بالا نمی‌آید.

۴.۳ Secret و ConfigMap

هر تنظیم غیرحساس (مثل لیست تیم‌ها یا ورزشگاه‌ها در داده‌ی پایه‌ی سرویس‌ها) را در `ConfigMap` و هر مقدار حساس (مثل گذرواژه‌ی `root` مربوط به `GitLab` یا توکن `Runner`) را در `Secret` نگه دارید. استفاده از `Secret` برای توکن `Runner` در بخش ۷ ضروری است.

۴.۴ نگه‌داری لاگ‌ها و خروجی‌ها با PersistentVolumeClaim

برای آن‌که لاگ تولیدشده توسط `Nginx` و سرویس‌ها در مرحله‌ی بعد توسط `MapReduce` خوانده شود، به یک حجم پایدار مشترک نیاز دارید:

نکته

روی کلاستر تک‌نودی (`kind/minikube`)، `StorageClass` پیش‌فرض (`standard` یا `local-path`) این `PVC` را به‌صورت خودکار `provision` می‌کند و چون همه‌ی `Pod`ها روی یک نود اجرا می‌شوند، `ReadWriteOnce` کافی است. اگر کلاستر چندنودی دارید و چند `Pod` هم‌زمان باید روی یک حجم بنویسند/بخوانند، به یک حجم `ReadWriteMany` (مثلاً `NFS`) نیاز خواهید داشت. روش نگه‌داری لاگ‌ها و خروجی‌ها را در گزارش توضیح دهید.

۵. تولید ترافیک و اجرای MapReduce روی Kubernetes

در فاز اول این مراحل با دستور دستی اجرا می‌شدند. در فاز دوم هر مرحله یک Kubernetes Job است تا بتوان آن را از داخل pipeline به‌صورت تکرارپذیر اجرا کرد.

۵.۱ تولید ترافیک با Job

ابزار تولید ترافیک به‌جای آدرس لوکال، به Service داخلی Nginx درخواست می‌فرستد (نه مستقیم به backend). همان‌طور که در فاز اول مبنی بر عبور همه‌ی ترافیک از Nginx این‌جا هم برقرار است.

```
apiVersion: batch/v1
kind: Job
metadata: { name: traffic-generator }
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: generator
          image: traffic-generator:1.0
          imagePullPolicy: IfNotPresent
          args: ["--nginx-url", "http://nginx-gateway:80",
                "--requests", "100000"]
```

توجه

traffic-generator فقط نقش کاربر آزمایشی را دارد و لاگ خودش مبنای تحلیل نیست. مبنای تحلیل همان nginx_access.log و لاگ‌های سرویس‌ها روی PVC است. اگر generator مستقیماً به backend درخواست بفرستد و Nginx دور زده شود، اجرای پروژه ناقص است.

۵.۲ اجرای Hadoop و pipeline MapReduce روی کلاستر

کلاستر ساده‌ی Hadoop فاز اول را درون یک ایمج بسته‌بندی کنید (یا از همان ایمج Hadoop Streaming فاز اول استفاده کنید) و اجرای pipeline را در قالب یک Job تعریف کنید. این Job اسکریپت run_mapreduce.sh را اجرا می‌کند که همان پنج job اجباری فاز اول را به‌ترتیب روی Hadoop Streaming اجرا می‌کند و خروجی هر مرحله را ورودی مرحله‌ی بعد قرار می‌دهد.

```

apiVersion: batch/v1
kind: Job
metadata: { name: mapreduce-pipeline }
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: hadoop
          image: hadoop-streaming:3.2.1 # پروژه Hadoop + mapper/reducer شامل
          command: ["bash", "/project/scripts/run_mapreduce.sh"]
          volumeMounts:
            - { name: data, mountPath: /project/data } # ورودی: لاگ‌ها
            - { name: data, mountPath: /project/outputs, subPath: outputs }
      volumes:
        - name: data
          persistentVolumeClaim: { claimName: project-data }

```

نکته

استفاده از Pandas یا یک اسکریپت ساده‌ی Python به جای Hadoop Streaming کافی نیست؛ خروجی‌ها باید از اجرای واقعی mapper و reducer در جریان Hadoop Streaming تولید شوند. اگر می‌خواهید HDFS واقعی داشته باشید، می‌توانید namenode و datanode را به صورت یک StatefulSet کوچک تعریف کنید و در run_mapreduce.sh ابتدا فایل‌ها را با `hdfs dfs -put` وارد HDFS کنید؛ برای محدوده‌ی این درس، اجرای تک‌نودی Hadoop Streaming درون یک Job نیز قابل قبول است.

نتیجه‌ی مورد انتظار

پس از اجرای موفق این دو Job، باید فایل `outputs/final/summary.json` و همه‌ی خروجی‌های میانی `job1` تا `job5` روی PVC ساخته شده باشند و با `kubectl logs job/mapreduce-pipeline` اجرای موفق هر مرحله دیده شود.

۶. راه‌اندازی GitLab روی Kubernetes

مخزن پروژه باید درون GitLab قرار گیرد که خود گروه آن را در Kubernetes بالا آورده است.

۶.۱ منابع و ملاحظات

GitLab CE سنگین است و در عمل به حدود ۴ تا ۶ گیگابایت RAM نیاز دارد. پیش از شروع مطمئن شوید کلاستر شما (مثلاً minikube start --memory=۸۱۹۲) منابع کافی دارد. بالا آمدن کامل GitLab ممکن است چند دقیقه طول بکشد؛ تا سبز شدن readinessProbe صبر کنید.

توجه: در صورت محدودیت سخت‌افزاری، استفاده از نمونه سبک‌تر GitLab، کاهش منابع سرویس‌ها یا استفاده از یک ماشین مشترک بین اعضای گروه مجاز است.

۶.۲ Service و Deployment

فایل‌های deployment و service مناسب برای gitlab را نوشته به طوری که روی مرورگر روی پورت ۸۹۲۹ باز شود

۶.۳ دسترسی، ورود اولیه و ساخت مخزن

پس از آماده‌شدن GitLab، گذرواژه‌ی اولیه‌ی کاربر root را از داخل کانتینر بخوانید:

```
kubectl exec deploy/gitlab -- cat /etc/gitlab/initial_root_password
```

سپس از مرورگر هاست به `http://localhost:8929` وارد شوید (به‌خاطر نگاشت `NodePort 30880→8929`)، یک پروژه‌ی جدید بسازید و مخزن فاز اول/دوم را به آن `push` کنید. از داخل کلاستر، آدرس `origin` برابر است با:

```
http://gitlab.cc-project.svc.cluster.local/root/<project-name>.git
```

توجه

داشتن صرف یک نمونه GitLab کافی نیست؛ در جلسه‌ی ارائه باید پروژه‌ی واقعی، تنظیمات CI/CD و اجرای pipeline درون همین GitLab نشان داده شود.

سرویس‌های داخل Kubernetes نباید برای دسترسی به GitLab از `localhost` استفاده کنند و باید از DNS داخلی (`*.svc.cluster.local`) استفاده شود.

۷. راه‌اندازی GitLab Runner روی Kubernetes

Runner همان جزئی است که jobهای pipeline را اجرا می‌کند و باید درون کلاستر اجرا و به همان GitLab پروژه متصل شود.

۷.۱ نصب Runner

ساده‌ترین راه، نصب با Helm chart رسمی است:

```
helm repo add gitlab https://charts.gitlab.io
helm repo update
helm install gitlab-runner gitlab/gitlab-runner -n cc-project \
  --set gitlabUrl=http://gitlab.cc-project.svc \
  --set runnerToken=<RUNNER_AUTH_TOKEN> \
  --set rbac.create=true
```

نکته

در نسخه‌های جدید GitLab، روش توصیه‌شده «Runner authentication token» است: در GitLab به مسیر Settings → CI/CD → Runners بروید، یک «New project runner» بسازید و توکن تولیدشده را به‌جای registration token استفاده کنید. این توکن را در یک Secret نگه دارید و در دستور بالا یا در config.toml ارجاع دهید.

در صورت مشکل با helm chart می‌توانید از image آن استفاده کنید

۷.۲ نوع executor Kubernetes

Runner باید با executor نوع kubernetes پیکربندی شود تا هر job داخل یک Pod موقت در همان کلاستر اجرا شود. بخش کلیدی config.toml:

```
[[runners]]
  name = "k8s-runner"
  url = "http://gitlab.cc-project.svc"
  executor = "kubernetes"
  [runners.kubernetes]
    namespace = "cc-project"
    image = "alpine:3.19"
    privileged = false
```

```
service_account = "gitlab-runner"
```

نتیجه‌ی مورد انتظار

پس از ثبت موفق، در صفحه‌ی Runners پروژه باید Runner با وضعیت سبز (online) دیده شود و با اجرای اولین pipeline، Pod های موقت اجرای job در namespace پروژه ظاهر و سپس حذف شوند.

۸. طراحی Pipeline در GitLab CI/CD

pipeline باید کل مسیر را خودکار کند: از تست منطق parser/mapper/reducer تا انتشار گزارش نهایی.

۸.۱ مراحل (stages) مورد انتظار

test: تست‌های ساده برای منطق parser, mapper و reducer؛

build: ساخت ایمج سرویس‌ها و ابزارهای پروژه؛

deploy: اعمال یا به‌روزرسانی manifestها روی Kubernetes؛

traffic: اجرای سناریوی ترافیک و تولید لاگ؛

mapreduce: اجرای هر پنج job اجباری MapReduce

validate: اعتبارسنجی خروجی MapReduce؛

report: تولید گزارش نهایی؛

publish: انتشار گزارش/داشبورد به‌صورت قابل مشاهده.

توجه: الزامی نیست که هر Push باعث اجرای کامل Pipeline شود. گروه‌ها می‌توانند از Stage های مجزا، Manual Job یا Trigger های مناسب برای جلوگیری از اجرای غیرضروری مراحل زمان‌بر استفاده کنند.

۸.۲ ساخت ایمج داخل کلاستر (Kaniko)

چون Runner با executor نوع kubernetes اجرا می‌شود و معمولاً دسترسی privileged به daemon داکر ندارید،

برای ساخت ایمج از Kaniko استفاده کنید که بدون نیاز به Docker daemon ایمج می‌سازد و در صورت داشتن

رجیستری آن را push می‌کند. اگر رجیستری ندارید، می‌توانید با kaniko --no-push و tarPath-- ایمج را بسازید و

سپس به کلاستر بارگذاری کنید؛ ساده‌ترین مسیر، راه‌اندازی یک رجیستری سبک داخل کلاستر است.

توجه: استفاده از Kaniko توصیه می‌شود. در صورت استفاده از روش‌های دیگر برای ساخت Image در

Kubernetes، گروه باید دلیل انتخاب و نحوه پیاده‌سازی خود را توضیح دهد

۸.۳ نمونه‌ی `gitlab-ci.yml`

```
stages: [test, build, deploy, traffic, mapreduce, validate, report, publish]

variables:
  NS: cc-project
  REGISTRY: registry.cc-project.svc:5000

test:
  ---

build-match:
  ---

deploy:
  ---

traffic:
  ---

mapreduce:
  ---

validate:
  ---

publish:
  ---
```

نکته

مراحل `validate` و `report` به محتوای روی `PVC` نیاز دارند. چون هر `job` در یک `Pod` جدا اجرا می‌شود، این مراحل را به‌صورت `Kubernetes Job`‌ی که همان `PVC project-data` را `mount` می‌کند پیاده کنید (`validate-output.yaml`). اسکریپت اعتبارسنجی باید وجود `summary.json`، درست‌بودن `JSON` و وجود کلیدهای اجباری مثل `total_requests` و `popular_team_by_country` را بررسی کند.

۸.۴ اعتبارسنجی و تولید گزارش

خروجی نهایی `summary.json` باید به یک صفحه‌ی قابل مشاهده تبدیل شود. می‌توانید در مرحله‌ی `report` یک `index.html` بسازید که `summary.json` را می‌خواند و آمار کلیدی (تعداد کل درخواست‌ها، پرتراфик‌ترین سرویس، نرخ خطا و محبوب‌ترین تیم هر کشور) را نمایش می‌دهد، و سپس آن را به عنوان `artifact` نگه دارید و در مرحله‌ی `publish` منتشر کنید.

اعتبارسنجی می‌تواند با یک اسکریپت ساده `Python` یا `Bash` انجام شود و نیاز به استفاده از ابزارهای پیچیده تست وجود ندارد.

توجه

صرف داشتن فایل `gitlab-ci.yml` کافی نیست. در جلسه‌ی ارائه باید اجرای واقعی `pipeline`، وضعیت `Runner`، لاگ `job`ها، `artifact`ها و خروجی `deploy` شده نشان داده شود.

۹. خروجی deploy شده

خروجی نهایی پروژه باید قابل مشاهده باشد و صرف یک فایل محلی بدون deploy، تحویل کامل محسوب نمی‌شود. خروجی می‌تواند یکی از این‌ها باشد: یک صفحه‌ی HTML خلاصه‌ی تحلیل، یک سرویس وب ساده که فایل‌های خروجی MapReduce را نمایش می‌دهد، یا یک داشبورد ساده‌ی تعداد درخواست‌ها و نرخ خطا و موارد مشکوک.

نمونه‌ی ساده با یک Nginx که محتوای گزارش را serve می‌کند و از طریق NodePort در دسترس است:

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: report-web }
spec:
  replicas: 1
  selector: { matchLabels: { app: report-web } }
  template:
    metadata: { labels: { app: report-web } }
    spec:
      containers:
        - name: web
          image: nginx:1.27-alpine
          volumeMounts:
            - { name: data, mountPath: /usr/share/nginx/html, subPath: outputs/final }
      volumes:
        - name: data
          persistentVolumeClaim: { claimName: project-data }
---
apiVersion: v1
kind: Service
metadata: { name: report-web }
spec:
  type: NodePort
  selector: { app: report-web }
  ports: [{ port: 80, targetPort: 80, nodePort: 30090 }]
```

نتیجه‌ی مورد انتظار

با نگاشت 9000→30090 NodePort، خروجی باید از روی مرورگر هاست در آدرس

<http://localhost:9000/index.html> و صفحه‌ی داشبورد <http://localhost:9000/summary.json>

قابل مشاهده باشد. خروجی باید از طریق Service/NodePort/Ingress در دسترس باشد، نه فقط به صورت فایل محلی.

هدف این بخش، قابل مشاهده بودن نتایج است و انتظار طراحی رابط کاربری پیشرفته یا گرافیکی وجود ندارد.

۱۰. اجرای بخش امتیازی Spark روی Kubernetes (اختیاری)

اگر بخش امتیازی Spark Structured Streaming را انجام می‌دهید، توصیه می‌شود مسیر streaming نیز با معماری Kubernetes سازگار باشد. ساده‌ترین راه، اجرای برنامه‌ی Spark در قالب یک Deployment یا Job درون همان namespace است که پوشه‌ی ورودی stream را از PVC مشترک می‌خواند و خروجی زنده تولید می‌کند. اجرای Spark خارج از Kubernetes هم قابل قبول است، مشروط بر آن که نحوه‌ی اتصال آن به سامانه‌ی اصلی به طور شفاف توضیح داده شود.

نکته

این بخش امتیازی است و جایگزین فاز اول یا فاز دوم نمی‌شود. حتی در صورت انجام آن، پنج job اجباری MapReduce و مسیر GitLab CI/CD همچنان اجباری‌اند.

۱۱. ملاحظات جلسه‌ی ارائه و Demo

سخت‌گیری اصلی در ارائه روی این است که مشخص شود پروژه واقعاً توسط اعضای گروه فهمیده و اجرا شده است. صرف وجود کد یا خروجی آماده برای نمره‌ی کامل کافی نیست. در جلسه باید موارد زیر به‌صورت زنده نشان داده و توضیح داده شوند:

وضعیت Pod ها و Service ها با `kubectl get pods,svc -n cc-project`؛

وضعیت online بودن GitLab Runner و اجرای واقعی یک pipeline از ابتدا تا انتها؛

لاگ job های artifact، pipeline ها و خروجی deploy شده؛

ایجاد یک تغییر کوچک در سناریوی ترافیک و نشان دادن این‌که خروجی تحلیل تغییر می‌کند؛

توضیح نقش mapper و reducer در یکی از job ها توسط هر عضو؛

بررسی history کامیت و مشارکت هر دو عضو در بخش‌های اصلی پروژه.

توجه

اگر مشخص شود گروه نمی‌تواند منطق پروژه، تنظیمات Kubernetes، مسیر CI/CD یا عملکرد MapReduce را توضیح دهد، حتی با وجود خروجی ظاهراً درست، نمره‌ی کامل داده نخواهد شد.

۱۲. موارد تحویلی فاز دوم

مورد تحویلی

موارد زیر باید در تحویل فاز دوم وجود داشته باشند:

لینک یا فایل پشتیبان مخزن GitLab راه‌اندازی شده توسط گروه؛

همه‌ی فایل‌های Kubernetes: Deployment/Service سرویس‌ها و Nginx، ConfigMap/Secret، Job ها

(تولید ترافیک، MapReduce، اعتبارسنجی)، PVC و manifest مربوط به GitLab و Runner؛

فایل gitlab-ci.yml و توضیح کوتاه مراحل pipeline؛

خروجی اجرای artifact، pipeline ها و گزارش نهایی؛

لینک، آدرس محلی یا تصویر خروجی deploy شده (داشبورد/گزارش)؛

گزارش کوتاه شامل معماری Kubernetes، نقش GitLab و Runner، ساختار CI/CD و نتیجه‌ی اجرا؛

در صورت انجام بخش امتیازی، کد و تصویر/ویدئوی اجرای زنده‌ی Spark.

۱۳. معیار نمره‌دهی پیشنهادی فاز دوم

نمره	بخش
۲۰	استقرار سرویس‌ها و Nginx روی Kubernetes (Deployment/Service/ConfigMap/PVC)
۲۵	راه‌اندازی GitLab و GitLab Runner درون Kubernetes
۳۰	طراحی pipeline ی GitLab CI/CD شامل test, build, deploy, traffic, mapreduce, validate و publish
۱۵	کیفیت، اعتبارسنجی و قابل مشاهده بودن خروجی deploy شده
۱۰	demo زنده، پاسخ به سؤال‌ها و مشارکت هر دو عضو
تا ۱۵+	بخش امتیازی Spark Structured Streaming سازگار با Kubernetes

۱۴. نکات مهم

GitLab و GitLab Runner باید واقعاً درون Kubernetes راه‌اندازی شوند؛ استفاده از GitLab.com به‌تنهایی کافی نیست.

اجرای پروژه فقط با Docker Compose کافی نیست؛ بخش اصلی باید با Kubernetes مدیریت شود.

اجرای کامل دستی بدون GitLab CI/CD کافی نیست؛ مسیر ساخت تا انتشار باید خودکار باشد.

پنج job اجباری MapReduce همچنان پابرجاست و با بخش امتیازی جایگزین نمی‌شود.

خروجی نهایی باید deploy شده و در جلسه‌ی ارائه قابل مشاهده باشد.

در صورت محدودیت سخت‌افزاری، استفاده از نمونه سبک‌تر GitLab، کاهش منابع سرویس‌ها یا استفاده از یک ماشین مشترک بین اعضای گروه مجاز است.

فایل‌ها و کدهای موجود در این فایل صرفاً مثال هستند و می‌توانند کار نکنند، پس می‌توانید طبق فایل‌ها و کدهای خودتان آنها را تغییر دهید.

توجه

اگر GitLab Runner یا سرویس‌های پروژه توسط Kubernetes مدیریت نشده باشند، پروژه از هدف فاز دوم فاصله گرفته و نمره‌ی کامل این فاز را دریافت نخواهد کرد.